# INTELIGÊNCIA ARTIFICIAL NA DETECÇÃO DE MALÁRIA: ESTUDO COMPARATIVO DE REDES NEURAIS CONVOLUCIONAIS EM IMAGENS DE CÉLULAS SANGUÍNEAS

CARVALHO, Alejandro[1]
KRETSCHMANN, Ângela[2]
MARTINS, Giliard[3]

Orientador: BRUSTOLIN, Gian[4]

## RESUMO

A aplicação de técnicas de Inteligência Artificial tem ampliado as possibilidades de automatização da análise de imagens biológicas. Este trabalho compara arquiteturas de Redes Neurais Convolucionais (*Convolutional Neural Networks* – CNNs) na classificação de imagens de células sanguíneas quanto à infecção por malária. A pesquisa, de natureza aplicada e quantitativa, utiliza o conjunto de dados público "*Cell Images for Detecting Malaria*", com 27.558 imagens balanceadas entre células parasitadas e não parasitadas. Foram implementadas, em Python com TensorFlow/Keras, uma CNN própria e três modelos pré-treinados com transferência de aprendizado (VGG16, ResNet50 e EfficientNetB0), treinados sob as mesmas condições e avaliados por acurácia, precisão, *recall* e *F1-score*. No conjunto de validação, a CNN própria obteve o melhor desempenho (*F1-score* de 95,81%), superando os modelos pré-treinados, o que indica que arquiteturas mais complexas não asseguram, por si sós, melhor resultado. Por terem sido obtidos no conjunto de validação, os resultados são preliminares e devem ser confirmados com um conjunto de teste independente.

**Palavras-chave:** Inteligência Artificial; Redes Neurais Convolucionais; Malária; Classificação de Imagens; Transferência de Aprendizado.

## 1 INTRODUÇÃO

A aplicação de técnicas de Inteligência Artificial (IA) e de processamento de imagens tem se consolidado como uma solução eficiente para problemas que exigem análises rápidas e precisas, especialmente na área da saúde. O diagnóstico da malária por microscopia, por exemplo, depende da inspeção manual de lâminas de sangue, processo que demanda elevado tempo de execução e é suscetível a falhas humanas e inconsistências nos resultados.

Diante desse cenário, este trabalho tem como objetivo estudar e comparar diferentes arquiteturas de Redes Neurais Convolucionais, com o intuito de identificar

[1]Graduando em Engenharia de Software pela UNINTER. RU: 4547278.
[2]Graduanda em Engenharia de Software pela UNINTER. RU: 4110110.
[3]Graduando em Engenharia de Software pela UNINTER. RU: 4410793.
[4]M.Sc., professor orientador.

aquela que apresenta melhor desempenho na classificação de imagens de células sanguíneas quanto à infecção por malária. A partir dessa proposta, busca-se responder à seguinte questão de pesquisa: qual arquitetura de Rede Neural Convolucional, entre uma CNN própria, a VGG16, a ResNet50 e a EfficientNetB0, apresenta melhor desempenho na identificação de células parasitadas?

Os objetivos específicos são: (i) revisar conceitos relacionados à Inteligência Artificial e visão computacional; (ii) selecionar e preparar um conjunto de dados adequado ao problema estudado; (iii) implementar uma CNN própria e os modelos pré-treinados VGG16, ResNet50 e EfficientNetB0; (iv) avaliar os modelos por meio de métricas de desempenho; (v) comparar os resultados obtidos e identificar a abordagem mais eficiente.

Como hipótese de trabalho, considera-se que diferentes arquiteturas de Redes Neurais Convolucionais podem apresentar variações relevantes de desempenho na detecção de células parasitadas, e que modelos pré-treinados mais complexos nem sempre superam uma CNN mais simples, sendo possível identificar, por meio das métricas de avaliação adotadas, a arquitetura mais eficiente para o conjunto de dados analisado.

A relevância desta pesquisa está associada ao potencial de utilização da Inteligência Artificial na automatização de análises laboratoriais, como o apoio à triagem de lâminas de sangue, contribuindo para a redução de erros humanos e para o aumento da confiabilidade dos resultados. Além disso, o estudo envolve o desenvolvimento de um *pipeline* de classificação reprodutível e versionado em repositório público, o que o aproxima das práticas da Engenharia de Software.

Além desta introdução, o artigo apresenta a fundamentação teórica (Seção 2), a metodologia (Seção 3), os resultados e sua discussão (Seção 4) e as considerações finais (Seção 5).

## 2 FUNDAMENTAÇÃO TEÓRICA

A detecção automatizada de padrões em imagens biológicas depende da integração entre a visão computacional e o aprendizado profundo. Nesse contexto, esta seção reúne os conceitos de Inteligência Artificial, Processamento Digital de

Imagens e Redes Neurais Convolucionais, além das métricas e ferramentas utilizadas para a construção e validação dos modelos de classificação.

## 2.1 INTELIGÊNCIA ARTIFICIAL E APRENDIZADO DE MÁQUINA

De acordo com Faceli et al. (2011) e Monard e Baranauskas (2003), o aprendizado de máquina permite que sistemas computacionais identifiquem padrões e tomem decisões com base em dados históricos. Segundo Mitchell (1997), um programa computacional aprende quando sua performance em determinada tarefa melhora com a experiência adquirida por meio dos dados. Segundo Goodfellow, Bengio e Courville (2016), a IA, especialmente por meio do aprendizado de máquina, permite que sistemas computacionais aprendam a partir de dados, sem a necessidade de programação explícita para cada tarefa.

## 2.2 PROCESSAMENTO DIGITAL DE IMAGENS

No contexto da análise de imagens, destaca-se a área de visão computacional, que tem como objetivo permitir que máquinas interpretem e compreendam informações visuais de maneira semelhante ao ser humano. De acordo com Gonzalez e Woods (2010), o processamento digital de imagens envolve técnicas de manipulação e análise de imagens digitais com o intuito de extrair informações relevantes para aplicações diversas, incluindo diagnósticos médicos e análises laboratoriais.

A aplicação dessas técnicas na área biológica tem se mostrado particularmente promissora, especialmente na análise de imagens de células (LITJENS et al., 2017). Nesse cenário, métodos automatizados baseados em Inteligência Artificial surgem como alternativa à identificação manual de células parasitadas, que exige elevado nível de atenção e é suscetível a falhas humanas.

## 2.3 REDES NEURAIS CONVOLUCIONAIS

Dentre as abordagens mais utilizadas na classificação de imagens, destacam-se as Redes Neurais Convolucionais (CNNs), amplamente empregadas em tarefas de reconhecimento visual (KRIZHEVSKY; SUTSKEVER; HINTON, 2012). Segundo LeCun, Bengio e Hinton (2015), os avanços recentes em aprendizado profundo consolidaram as CNNs como uma das principais arquiteturas para processamento de imagens.

Essas redes são um tipo de arquitetura de aprendizado profundo projetada especificamente para o processamento de dados com estrutura em grade, como imagens. Segundo Goodfellow, Bengio e Courville (2016), as CNNs são compostas por camadas convolucionais, responsáveis pela extração de características; camadas de *pooling*, que reduzem a dimensionalidade dos dados; e camadas totalmente conectadas, utilizadas para a classificação final. Sua principal vantagem é extrair automaticamente características relevantes das imagens, dispensando a engenharia manual de atributos.

Além de arquiteturas projetadas especificamente para um problema, é comum o uso de transferência de aprendizado (*transfer learning*), em que redes previamente treinadas em grandes bases de imagens, como a ImageNet, são reaproveitadas como extratoras de características, e apenas as camadas finais são treinadas para a nova tarefa. Entre as arquiteturas pré-treinadas mais utilizadas destacam-se a VGG16, que empilha convoluções de 3 × 3 (SIMONYAN; ZISSERMAN, 2014); a ResNet50, que introduz conexões residuais (HE et al., 2016); e a EfficientNet, que escala de forma combinada a profundidade, a largura e a resolução da rede (TAN; LE, 2019).

Estudos recentes reforçam a aplicação de diferentes arquiteturas de Redes Neurais Convolucionais à classificação automatizada de células parasitadas por malária. Ukeje et al. (2026), ao compararem uma CNN convencional e uma arquitetura híbrida CNN-LSTM em 27.558 imagens microscópicas, obtiveram acurácia de 94,90% para a CNN, evidenciando que modelos relativamente leves podem apresentar desempenho competitivo. Laghari et al. (2025), por sua vez, avaliaram oito arquiteturas pré-treinadas, incluindo VGG, ResNet, Inception e Xception, verificando diferenças relevantes entre seus desempenhos. Já Mujahid et al. (2024) obtiveram acurácia de 97,57% com uma abordagem baseada em EfficientNet. Esses resultados

reforçam a pertinência de comparar diferentes arquiteturas sob condições experimentais equivalentes, uma vez que a complexidade do modelo, isoladamente, não determina seu desempenho.

## 2.4 PRÉ-PROCESSAMENTO DE IMAGENS

O pré-processamento constitui uma etapa relevante na aplicação de modelos de classificação a imagens digitais. Entre os procedimentos empregados estão o redimensionamento das imagens, a normalização dos valores de pixel e a aplicação de técnicas de aumento de dados (*data augmentation*), que podem contribuir para melhorar a capacidade de generalização dos modelos e reduzir problemas como o sobreajuste (*overfitting*).

Técnicas de processamento digital de imagens permitem a melhoria da qualidade das imagens e a extração de características relevantes (QUEIROZ; GOMES, 2006), contribuindo para a padronização dos dados fornecidos aos modelos.

## 2.5 MÉTRICAS DE AVALIAÇÃO DE MODELOS

Outro aspecto relevante na avaliação de modelos de classificação é a utilização de métricas de desempenho adequadas. Entre as principais métricas utilizadas destacam-se a acurácia, que mede a proporção de classificações corretas; a precisão, que avalia a proporção de verdadeiros positivos entre os resultados positivos; o *recall*, também denominado sensibilidade, que indica a capacidade do modelo de identificar corretamente os casos positivos; a especificidade, que indica a capacidade de identificar corretamente os casos negativos; e o *F1-score*, que representa a média harmônica entre precisão e *recall* (POWERS, 2011). Essas métricas são calculadas a partir da matriz de confusão, que registra os verdadeiros positivos (VP), os falsos negativos (FN), os falsos positivos (FP) e os verdadeiros negativos (VN).

Em aplicações de apoio ao diagnóstico, os falsos negativos são particularmente críticos, pois correspondem a células infectadas classificadas como saudáveis. Por esse motivo, neste trabalho a classe parasitada é tratada como positiva, e a sensibilidade deve receber atenção especial na análise dos resultados.

A adoção conjunta dessas métricas também favorece a comparação dos resultados deste estudo com pesquisas recentes sobre classificação automatizada de células parasitadas, que utilizam indicadores semelhantes para avaliar diferentes arquiteturas de aprendizado profundo (MUJAHID et al., 2024; UKEJE et al., 2026; LAGHARI et al., 2025).

## 3 METODOLOGIA

Esta pesquisa caracteriza-se como um estudo de natureza aplicada, com abordagem quantitativa. Quanto aos procedimentos técnicos, trata-se de uma pesquisa empírica de caráter experimental, que envolve o desenvolvimento de um software para a classificação automatizada de imagens celulares. O objetivo metodológico reside no estabelecimento de um fluxo de trabalho (*pipeline*) de aprendizado profundo (*deep learning*) capaz de processar dados visuais e classificar automaticamente as imagens quanto à infecção por malária.

A abordagem estrutura-se em quatro etapas principais: (1) seleção e organização do conjunto de dados e sua divisão em subconjuntos; (2) pré-processamento e aumento de dados; (3) construção e treinamento das arquiteturas de CNNs; e (4) avaliação e comparação dos modelos por meio das métricas definidas neste estudo.

### 3.1 FERRAMENTAS E LINGUAGENS

Os experimentos foram implementados na linguagem Python, amplamente utilizada em ciência de dados e aprendizado de máquina. Foram empregadas as bibliotecas TensorFlow/Keras, para a construção e o treinamento das redes; NumPy, para a manipulação numérica; scikit-learn, para o cálculo das métricas e da matriz de confusão (PEDREGOSA et al., 2011); Pandas, para a organização da tabela comparativa; e Matplotlib, para a geração dos gráficos.

O código-fonte e os gráficos de resultados estão versionados no repositório oficial do projeto, disponível em https://github.com/alecarvalho950/TCC (CARVALHO; KRETSCHMANN; MARTINS, 2026), que também registra o histórico das

contribuições de cada autor. As dependências foram instaladas em um ambiente virtual Python, e as instruções de instalação e execução estão documentadas no arquivo README do repositório. Perfis dos autores no GitHub: https://github.com/alecarvalho950, https://github.com/angelakretschmann e https://github.com/Gili10.

## 3.2 CONJUNTO DE DADOS

Foi selecionado o conjunto de dados público "*Cell Images for Detecting Malaria*", disponível na plataforma Kaggle (KAGGLE, 2018) e originado do trabalho de Rajaraman et al. (2018), da *National Library of Medicine* dos Estados Unidos. O conjunto é composto por imagens de células parasitadas e não parasitadas, que correspondem, neste estudo, às classes positiva e negativa, respectivamente.

## 3.3 PRÉ-PROCESSAMENTO E DIVISÃO DOS DADOS

A divisão do conjunto de dados é realizada pelo script dividir_dados.py, que separa as imagens em pastas de treinamento e de validação. Para cada classe, são sorteadas 4.134 imagens, o equivalente a 15% do total do conjunto, para a validação, com semente fixa (*seed* = 123), o que torna a divisão reprodutível. Dessa forma, 19.290 imagens (70%) são destinadas ao treinamento e 8.268 imagens (30%) à validação, com o mesmo número de imagens de cada classe no conjunto de validação. Nesta etapa, não foi separado um conjunto de teste independente, limitação discutida na Seção 4.3.

As imagens são carregadas por meio da função *image_dataset_from_directory*, do Keras, redimensionadas para 128 × 128 pixels e agrupadas em lotes (*batches*) de 32 imagens. Para a CNN própria, a VGG16 e a ResNet50, os valores dos pixels são normalizados para o intervalo [0, 1]; para a EfficientNetB0, mantém-se o intervalo original [0, 255], uma vez que essa arquitetura incorpora sua própria normalização. Durante o treinamento, aplica-se aumento de dados (*data augmentation*) por meio de rotações aleatórias de até ±36° e de inversões horizontais e verticais.

## 3.4 ARQUITETURAS IMPLEMENTADAS

Foram implementadas quatro arquiteturas. A CNN própria é composta por três blocos convolucionais, com 32, 64 e 128 filtros de 3 × 3 e ativação ReLU, cada um seguido de uma camada de *max pooling* 2 × 2, além de uma camada densa de 128 neurônios, *dropout* de 50% e uma camada de saída com ativação *softmax* para as duas classes. Os modelos VGG16, ResNet50 e EfficientNetB0 utilizam pesos pré-treinados na base ImageNet, com as camadas convolucionais congeladas; sobre cada base foram acrescentadas uma camada de *global average pooling*, uma camada densa de 128 neurônios, *dropout* de 50% e a camada de saída *softmax*.

Todos os modelos são treinados com o otimizador Adam, com taxa de aprendizado de 0,0001, e com a função de perda entropia cruzada categórica, por até 20 épocas. Utilizam-se os mecanismos de parada antecipada (*early stopping*), com paciência de cinco épocas sobre a perda de validação, e de salvamento do melhor modelo (*model checkpoint*), de modo a preservar a época com menor perda de validação.

## 3.5 AVALIAÇÃO DOS MODELOS

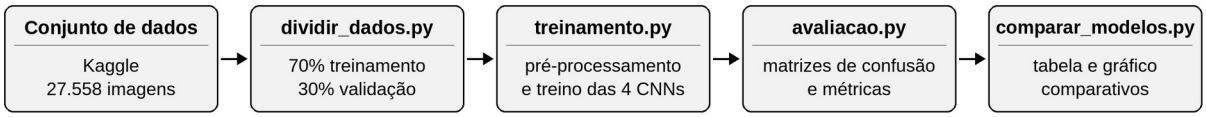
A avaliação é realizada por dois scripts. O primeiro (avaliacao.py) gera, para cada modelo, a matriz de confusão e as métricas de acurácia, sensibilidade (*recall*), especificidade, precisão e *F1-score*, considerando a classe parasitada como positiva. O segundo (comparar_modelos.py) consolida os resultados em uma tabela comparativa e em um gráfico de classificação dos modelos ordenado pelo *F1-score*; nessa consolidação, a precisão, o *recall* e o *F1-score* correspondem à média ponderada das duas classes.

## 3.6 ESTRUTURA DO SOFTWARE DESENVOLVIDO

O software foi organizado em quatro scripts Python, armazenados na pasta “codigos” do repositório do projeto e executados em sequência, conforme o fluxograma apresentado na Figura 1. O script dividir_dados.py realiza a divisão das imagens em treinamento e validação. O script treinamento.py aplica o pré-

processamento e treina sequencialmente as quatro arquiteturas, salvando o melhor modelo de cada uma e o gráfico de seu histórico de treinamento. O script avaliacao.py carrega os modelos salvos, gera as matrizes de confusão e calcula as métricas individuais. Por fim, o script comparar_modelos.py consolida as métricas de todos os modelos em uma tabela comparativa e em um gráfico de classificação.

Figura 1 – Fluxograma do software de classificação desenvolvido.




Fonte: Autoria própria (2026).

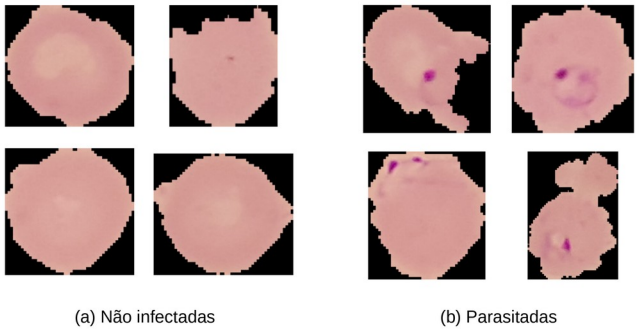
## 4 RESULTADOS E DISCUSSÕES

Esta seção apresenta a caracterização do conjunto de dados, os resultados das quatro arquiteturas e a discussão de suas limitações.

### 4.1 CARACTERIZAÇÃO DO CONJUNTO DE DADOS

O conjunto de dados possui 27.558 imagens, sendo 13.779 de células parasitadas (classe positiva) e 13.779 de células não parasitadas (classe negativa). Esse equilíbrio exato reduz o risco de o modelo favorecer uma das classes e favorece a interpretação das métricas de avaliação. A Figura 2 ilustra amostras do conjunto referentes às células (a) não infectadas e (b) infectadas (parasitadas).

Figura 2 – Exemplos de imagens de células (a) não infectadas e (b) parasitadas.



(a) Não infectadas (b) Parasitadas

Fonte: Adaptado de Kaggle (2018).

Enquanto as células não parasitadas apresentam estrutura e coloração predominantemente uniformes, as células parasitadas exibem manchas escuras

associadas à presença do parasita, padrão visual que as CNNs devem aprender a distinguir.

## 4.2 RESULTADOS DOS MODELOS

As quatro arquiteturas foram treinadas sob as mesmas condições, descritas nas Seções 3.3 e 3.4, e avaliadas sobre as 8.268 imagens do conjunto de validação. A Tabela 1 apresenta as métricas obtidas, e a Figura 3, o gráfico comparativo gerado pelo script comparar_modelos.py.

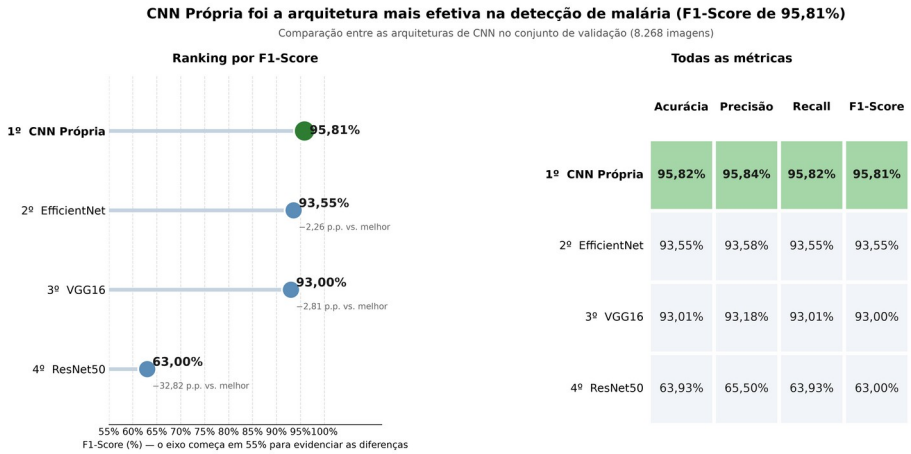
Tabela 1 – Métricas dos modelos no conjunto de validação (média ponderada das duas classes).

| **Modelo** | **Acurácia** | **Precisão** | ***Recall*** | ***F1-score*** |
|---|---|---|---|---|
| CNN própria | 95,82% | 95,84% | 95,82% | 95,81% |
| EfficientNetB0 | 93,55% | 93,58% | 93,55% | 93,55% |
| VGG16 | 93,01% | 93,18% | 93,01% | 93,00% |
| ResNet50 | 63,93% | 65,50% | 63,93% | 63,00% |

Fonte: Autoria própria (2026), a partir dos resultados do script comparar_modelos.py.

Por se tratar de médias ponderadas das duas classes, o *recall* coincide com a acurácia e não informa isoladamente a sensibilidade para a classe parasitada.

Figura 3 – Comparação geral das métricas de validação dos quatro modelos.



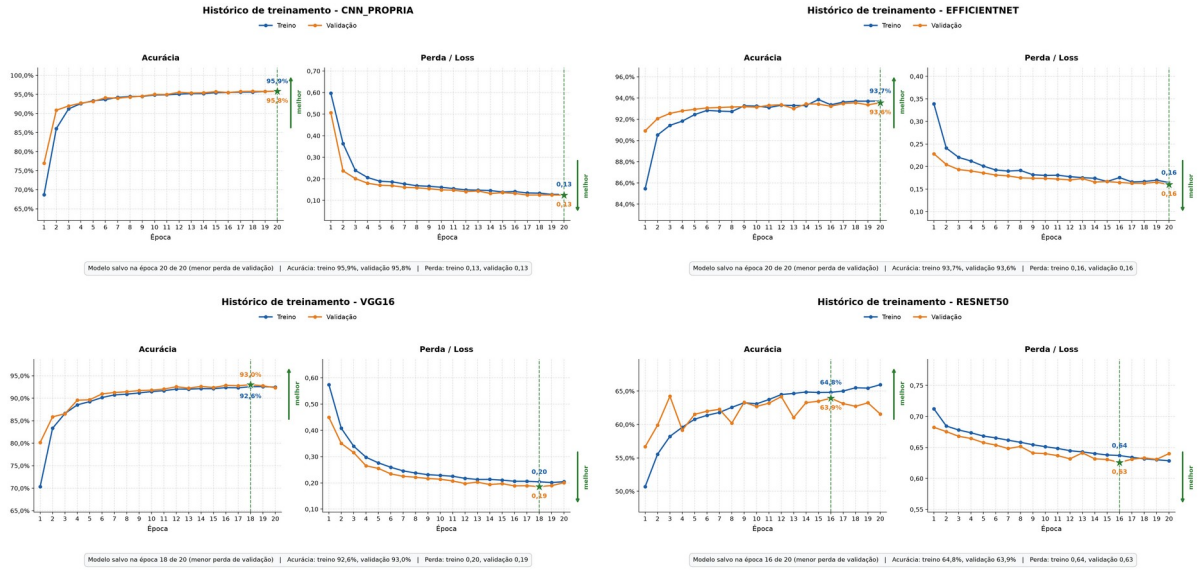

Fonte: Autoria própria (2026).

A CNN própria obteve o melhor desempenho em todas as métricas, com acurácia de 95,82% e *F1-score* de 95,81%. A EfficientNetB0 (93,55%) e a VGG16 (93,00%) formaram um segundo grupo, com desempenhos próximos entre si e 2,26 e 2,81 pontos percentuais, respectivamente, abaixo da CNN própria. A ResNet50

apresentou desempenho substancialmente inferior, com *F1-score* de 63,00%, 32,82 pontos percentuais abaixo do melhor modelo.

A Figura 4 apresenta as curvas de acurácia e de perda de cada modelo ao longo das épocas. O melhor modelo, correspondente à menor perda de validação, foi obtido na 20ª época para a CNN própria e a EfficientNetB0, na 18ª para a VGG16 e na 16ª para a ResNet50.

Figura 4 – Histórico de treinamento da CNN própria, da EfficientNetB0, da VGG16 e da ResNet50.




Fonte: Autoria própria (2026).

Nos três melhores modelos, as curvas de treino e de validação permanecem próximas, sem indício de sobreajuste. Na CNN própria e na EfficientNetB0, a menor perda de validação ocorreu na última época, o que sugere que um número maior de épocas ainda poderia produzir ganhos. Na ResNet50, ao contrário, a perda diminuiu lentamente, de 0,68 para 0,63 na validação, e a acurácia de validação oscilou entre aproximadamente 57% e 64%, o que caracteriza um modelo que não conseguiu aprender adequadamente a tarefa nas condições testadas. Os resultados sugerem que a CNN própria apresentou melhor adaptação às características específicas do conjunto de dados utilizado, alcançando desempenho superior aos modelos pré-treinados avaliados.

## 4.3 DISCUSSÃO DOS RESULTADOS E LIMITAÇÕES

Os resultados confirmam que arquiteturas diferentes apresentam desempenhos distintos no mesmo conjunto de dados, o que é coerente com a hipótese de trabalho. Uma explicação para a vantagem da CNN própria é que ela foi a única arquitetura treinada integralmente sobre o conjunto de dados, enquanto os modelos pré-treinados mantiveram suas camadas convolucionais congeladas; assim, maior complexidade e pesos pré-treinados não asseguram, por si sós, melhor desempenho. Além disso, esses modelos foram treinados originalmente na base ImageNet, composta por fotografias de objetos e animais, muito diferentes das imagens de microscopia, o que reduz o aproveitamento das características aprendidas quando suas camadas permanecem congeladas.

O desempenho da CNN própria (95,82% de acurácia) é superior ao relatado por Ukeje et al. (2026) para uma CNN convencional no mesmo conjunto de dados (94,90%) e inferior ao obtido por Mujahid et al. (2024) com uma abordagem baseada em EfficientNet (97,57%). Esses valores estão na faixa esperada para o conjunto, ressalvadas as diferenças de protocolo entre os estudos.

O resultado da ResNet50 merece atenção particular. Uma explicação provável é o pré-processamento das imagens: a ResNet50 e a VGG16 foram treinadas originalmente com uma normalização própria dos pixels, distinta da escala [0, 1] utilizada neste estudo, e, com a base congelada, não podem adaptar suas camadas a essa diferença, efeito que foi mais intenso na ResNet50. A EfficientNetB0, que recebeu as imagens no formato esperado, obteve desempenho próximo ao da CNN própria. Desse modo, as condições experimentais favoreceram a CNN própria, e uma comparação definitiva requer o pré-processamento específico e o ajuste fino (*fine-tuning*) dos modelos pré-treinados.

Os resultados devem ser interpretados com cautela, por três razões. Primeiro, o conjunto de validação foi utilizado tanto para selecionar a melhor época de cada modelo quanto para calcular as métricas, o que tende a produzir estimativas otimistas; uma avaliação definitiva requer um conjunto de teste independente. Segundo, o conjunto de dados contém várias células provenientes de um mesmo paciente, de modo que a divisão aleatória pode distribuir células de um mesmo indivíduo entre treinamento e validação, favorecendo o desempenho medido. Terceiro, cada modelo

foi treinado uma única vez, sem repetições com sementes diferentes que permitam avaliar a variabilidade dos resultados. A análise dos falsos negativos, críticos segundo a Seção 2.5, requer ainda as matrizes de confusão de cada modelo.

## 5 CONSIDERAÇÕES FINAIS

O trabalho resultou no desenvolvimento de um software que realiza a divisão dos dados, o treinamento, a avaliação e a comparação de quatro arquiteturas de Redes Neurais Convolucionais, disponível em repositório público com instruções para a reprodução dos experimentos.

Em relação ao objetivo geral, a CNN própria apresentou o melhor desempenho no conjunto de validação, com acurácia de 95,82% e *F1-score* de 95,81%, seguida pela EfficientNetB0 (93,55%) e pela VGG16 (93,00%). A ResNet50 obteve desempenho substancialmente inferior (63,00%), o que indica a necessidade de revisar seu pré-processamento antes da comparação definitiva.

No que se refere à hipótese de trabalho, os resultados indicam variações relevantes de desempenho entre as arquiteturas e mostram que modelos pré-treinados mais complexos nem sempre superam uma CNN mais simples, em consonância com a literatura revisada. Contudo, como as métricas foram obtidas no conjunto de validação, a confirmação definitiva da hipótese depende da avaliação em um conjunto de teste independente.

Como continuidade, recomenda-se avaliar os modelos em um conjunto de teste independente, adotar o pré-processamento específico e o ajuste fino (*fine-tuning*) dos modelos pré-treinados, repetir os treinamentos com sementes diferentes e incorporar as matrizes de confusão à análise.

## REFERÊNCIAS

CARVALHO, Alejandro; KRETSCHMANN, Ângela; MARTINS, Giliard. **TCC**: estudo comparativo de arquiteturas de redes neurais convolucionais para detecção de malária. GitHub, 2026. Disponível em: https://github.com/alecarvalho950/TCC. Acesso em: 26 out. 2026.

FACELI, Katti; LORENA, Ana Carolina; GAMA, João; CARVALHO, André C. P. L. F. de. **Inteligência Artificial**: uma abordagem de aprendizado de máquina. Rio de Janeiro: LTC, 2011.

GONZALEZ, Rafael C.; WOODS, Richard E. **Processamento Digital de Imagens**. 3. ed. São Paulo: Pearson, 2010.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep Learning**. Cambridge: MIT Press, 2016.

HE, Kaiming; ZHANG, Xiangyu; REN, Shaoqing; SUN, Jian. Deep residual learning for image recognition. In: IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION, 2016, Las Vegas. **Proceedings** [...]. Las Vegas: IEEE, 2016. p. 770-778.

KAGGLE. Cell Images for Detecting Malaria. **Kaggle**, 2018. Disponível em: https://www.kaggle.com/datasets/iarunava/cell-images-for-detecting-malaria. Acesso em: 15 abr. 2026.

KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HINTON, Geoffrey E. ImageNet classification with deep convolutional neural networks. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 25., 2012, Lake Tahoe. **Anais** [...]. Lake Tahoe: NIPS, 2012. p. 1097-1105.

LAGHARI, Azhar Ali; MUHAMMAD, Wazir; MEMON, Mudasar Latif; HUSSAIN, Ayaz; KUMAR, Akash. Malaria parasite cell classification using transfer learning with state-of-the-art CNN architectures. **Biology**, v. 14, n. 12, art. 1792, 2025. DOI: 10.3390/biology14121792.

LECUN, Yann; BENGIO, Yoshua; HINTON, Geoffrey. Deep learning. **Nature**, v. 521, n. 7553, p. 436-444, 2015.

LITJENS, Geert et al. A survey on deep learning in medical image analysis. **Medical Image Analysis**, v. 42, p. 60-88, 2017.

MITCHELL, Tom M. **Machine Learning**. New York: McGraw-Hill, 1997.

MONARD, Maria Carolina; BARANAUSKAS, José Augusto. Conceitos sobre aprendizado de máquina. In: REZENDE, Solange Oliveira (org.). **Sistemas inteligentes**: fundamentos e aplicações. Barueri: Manole, 2003. cap. 4.

MUJAHID, Muhammad; RUSTAM, Furqan; SHAFIQUE, Rahim et al. Efficient deep learning-based approach for malaria detection using red blood cell smears. **Scientific Reports**, v. 14, art. 13249, 2024. DOI: 10.1038/s41598-024-63831-0.

PEDREGOSA, Fabian et al. Scikit-learn: machine learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825-2830, 2011.

POWERS, David M. W. Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation. **Journal of Machine Learning Technologies**, v. 2, n. 1, p. 37-63, 2011.

QUEIROZ, José Eustáquio Rangel de; GOMES, Herman Martins. Introdução ao processamento digital de imagens. **RITA – Revista de Informática Teórica e Aplicada**, v. 13, n. 2, p. 11-42, 2006.

RAJARAMAN, Sivaramakrishnan et al. Pre-trained convolutional neural networks as feature extractors toward improved malaria parasite detection in thin blood smear images. **PeerJ**, v. 6, e4568, 2018. DOI: 10.7717/peerj.4568.

SIMONYAN, Karen; ZISSERMAN, Andrew. Very deep convolutional networks for large-scale image recognition. **arXiv**, 1409.1556, 2014. Disponível em: https://arxiv.org/abs/1409.1556. Acesso em: 26 out. 2026.

TAN, Mingxing; LE, Quoc V. EfficientNet: rethinking model scaling for convolutional neural networks. In: INTERNATIONAL CONFERENCE ON MACHINE LEARNING, 36., 2019, Long Beach. **Proceedings** [...]. PMLR, 2019. v. 97, p. 6105-6114.

UKEJE, Ndidiamaka; OMOEBAMIJE, Oluwaseun; USMAN, Mistura M.; OGWUELEKA, Francisca; IFECHELOBI, Chukwuemeka. Performance comparison of a hybrid convolutional neural network-long short-term memory and CNN model for malaria diagnosis using microscopic blood smears. **Applied Medical Informatics**, v. 48, n. 2, 2026. Disponível em: https://ami.info.umfcluj.ro/index.php/AMI/article/view/1255. Acesso em: 26 out. 2026.